



INFORME DE GUÍA PRÁCTICA #3

I. PORTADA

Tema:	Arboles
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero B
Alumnos participantes:	Nata Analuiza Walter Alexis
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar e implementar soluciones en C++ y Java para la manipulación y transformación de árboles N-arios y árboles binarios, aplicando conceptos fundamentales de estructuras de datos como inserción, recorridos, recursividad y modificaciones estructurales, con el fin de fortalecer las habilidades de programación y comprensión de estructuras jerárquicas.

Específicos:

- Implementar estructuras de árboles N-arios y árboles binarios utilizando C++ y Java para comprender su funcionamiento y organización jerárquica.
- Aplicar algoritmos de inserción en Árboles Binarios de Búsqueda (BST) para almacenar datos de manera ordenada y eficiente.
- Utilizar técnicas recursivas para calcular propiedades estructurales de los árboles, como la profundidad máxima y el conteo de nodos.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 4

No presenciales: 0

2.4 Instrucciones

1. Crear un proyecto en C++ y otro en Java para desarrollar cada uno de los ejercicios planteados en la práctica.
2. Revisar la estructura inicial proporcionada en cada archivo e identificar el método donde aparece el comentario TODO: Implementa tu lógica aquí.
3. Implementar únicamente la lógica solicitada en cada ejercicio sin modificar la estructura principal del programa ni el método main.
4. Resolver cada ejercicio según lo que nos pidan.
5. Ejecutar cada programa y verificar que los resultados obtenidos coincidan con las salidas esperadas mostradas en consola.
6. Organizar y documentar correctamente el código para facilitar su comprensión y mantenimiento.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- Computador, Cuadernos, esferos, lápiz, borrador, Aula virtual e Internet



TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☒ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial
- Otros (Especifique): _____

2.6 Actividades por desarrollar

- 2.6.1 Investigar los conceptos fundamentales relacionados con árboles N-arios, árboles binarios y Árboles Binarios de Búsqueda (BST).
- 2.6.2 Analizar la estructura base de los programas proporcionados en C++ y Java.
- 2.6.3 Implementar el algoritmo de conteo de nodos en árboles N-arios.
- 2.6.4 Desarrollar la lógica de inserción de datos en un Árbol Binario de Búsqueda.
- 2.6.5 Aplicar recursividad para calcular la profundidad máxima de un árbol binario.
- 2.6.6 Implementar el recorrido In-Order para visualizar los elementos almacenados en el árbol.
- 2.6.7 Realizar la transformación de un árbol binario mediante la inversión o generación del árbol espejo.
- 2.6.8 Ejecutar pruebas de funcionamiento en cada ejercicio para validar los resultados esperados.
- 2.6.9 Corregir errores de lógica o sintaxis encontrados durante la ejecución de los programas.
- 2.6.10 Documentar y organizar el código desarrollado en ambos lenguajes de programación.

2.7 Resultados obtenidos

Arboles N-arios

Un árbol n-ario es una estructura recursiva, en la cual cada elemento tiene un número cualquiera de árboles n-arios asociados. Estos árboles corresponden a la generalización de un árbol binario. La diferencia radica en que esta estructura puede manejar múltiples sub árboles asociados a cada elemento, y no solamente 2, como en el caso de los árboles binarios.[1]

Arboles Binarios

Un árbol binario es una estructura de datos donde cada nodo tiene hasta dos hijos, facilitando la implementación de algoritmos como búsqueda e inserción. Esta organización permite mantener los datos ordenados, lo que mejora la eficiencia de las búsquedas.[2]

Arboles binarios de búsqueda (BST)

Un árbol de búsqueda binaria (BST) es una estructura de datos que organiza los nodos de manera que, para cualquier nodo "X", todos los valores en su subárbol izquierdo son menores y todos los valores en su subárbol derecho son mayores. Esto permite realizar operaciones como búsqueda, inserción y eliminación de manera más eficiente en comparación con un árbol binario normal.[3]

Resolución de ejercicios:

1. Ejercicio 1: Árboles Básicos (Conteo de nodos en árboles N-arios).



En este ejercicio se implementó una función recursiva para contar la cantidad total de nodos en un árbol N-ario. El algoritmo recorre cada nodo del árbol y suma 1 por el nodo actual más el total de nodos de cada uno de sus hijos.

Código:

Hemos implementado esta función para la resolución del primer ejercicio, en donde la función va a recibir un puntero raíz, que es el nodo raíz del árbol. Tenemos una condicional en donde si la raíz es igual a NULL, significa que es árbol este vacío y retorna el 0.

Se inicia el total en 1 ya que se cuenta el nodo raíz, después se recorre toda la lista de nodos y se llama recursivamente a la función contarNodos() y se le suma al total. Finalmente me retorna el total de todos los nodos.

```
int contarNodos(NodoN* raiz) {  
    if (raiz == nullptr) {  
        return 0;  
    }  
  
    int total = 1;  
  
    for (NodoN* hijo : raiz->hijos) {  
        total += contarNodos(hijo);  
    }  
  
    return total;  
}
```

Ilustración 1. función contar Nodos

Resultado:

Ambos valores coinciden esto nos muestra que la función implementada esta funcionando correctamente.

```
--- Prueba Ejercicio 1 ---  
Nodos esperados: 6  
Nodos calculados: 6
```

Ilustración 2 Resultado

Se logró implementar correctamente el conteo de nodos en un árbol N-ario utilizando recursividad, obteniendo el resultado esperado durante la ejecución del programa.

Código JAVA

Hemos implementado de la misma manera la misma función, pero en java, en donde podemos observar cambia la sintaxis nada más, pero siguen la misma lógica



```
public static int contarNodos(NodoN raiz) {  
    if (raiz == null) {  
        return 0;  
    }  
  
    int total = 1;  
  
    for (NodoN hijo : raiz.hijos) {  
        total += contarNodos(hijo);  
    }  
  
    return total;  
}
```

Ilustración 3. Código java

Observamos que se utiliza NULL y los punteros en vez de -> utilizamos un (.). Los resultados son los mismo que se mostraron en la captura del código C++. También no utilizamos el puntero *, de C++ y en java solo se hace referencia al objeto.

2. Ejercicio 2: Árbol Binario (Inserción en BST).

En este ejercicio se implementó la lógica de inserción en un Árbol Binario de Búsqueda (BST).

El algoritmo compara el valor a insertar con el valor del nodo actual:

- Si el valor es menor, se inserta en el subárbol izquierdo.
- Si el valor es mayor, se inserta en el subárbol derecho.

La inserción se realiza utilizando recursividad hasta encontrar una posición vacía (nullptr).

Código:

Hemos implementado la función de inserción de un nodo en un árbol binario de búsqueda utilizando la recursividad. La función va a recibir un puntero raíz y el valor el cual vamos a insertar. Tenemos una condición el cual si el árbol esta vacío o una rama sin nodo, vamos a crear un nuevo nodo con un valor y retornamos.

Luego entramos a la nueva condición donde si el el valor es menor al valor de la raíz, vamos a insertar ese nodo a la izquierda.

En cambio, si es mayor el valor, vamos a insertarlo a la derecha. Hacemos utilidad de la recursividad para colocarlos en la posición correcta.

Finalmente, devuelve el nodo actual, el cual puede ser si mismo o un nuevo nodo



```
Nodo* insertar(Nodo* raiz, int valor) {  
  
    if (raiz == nullptr) {  
        return new Nodo(valor);  
    }  
  
    if (valor < raiz->valor) {  
        raiz->izquierdo = insertar(raiz->izquierdo, valor);  
    } else {  
        raiz->derecho = insertar(raiz->derecho, valor);  
    }  
  
    return raiz;  
}
```

Ilustración 4. Función de insertar nodo

Resultado:

Observamos que la función funciona correctamente, ya que ocupa cada nodo su lugar según los principios del árbol binario de búsqueda.

```
--- Prueba Ejercicio 2 ---  
Raiz (Esperado 10): 10  
Hijo Izquierdo (Esperado 5): 5  
Hijo Derecho (Esperado 15): 15  
Hijo Izq del 5 (Esperado 3): 3
```

Ilustración 5. Resultados

Se implementó correctamente la inserción de nodos en un Árbol Binario de Búsqueda utilizando recursividad, logrando organizar los valores según las propiedades del BST y obteniendo los resultados esperados en la ejecución del programa.

Código JAVA

Hemos implementado la misma función en java



```
public static Nodo insertar(Nodo raiz, int valor) {  
    if (raiz == null) {  
        return new Nodo(valor);  
    }  
  
    if (valor < raiz.valor) {  
        raiz.izquierdo = insertar(raiz.izquierdo, valor);  
    } else {  
        raiz.derecho = insertar(raiz.derecho, valor);  
    }  
  
    return raiz;  
}
```

Ilustración 6. Código JAVA

Como se implemento en C++, es la misma función cambia la sintaxis pero tenemos la misma lógica que es insertar un nodo. Utilizamos NULL en vez de NULLPTR, no usamos los punteros de C++ y hacemos referencia al objeto con (.).

3. Ejercicio 3: Árbol Binario (Cálculo de profundidad máxima).

En este ejercicio se implementó una función recursiva para calcular la altura o profundidad máxima de un árbol binario.

El algoritmo analiza la altura del subárbol izquierdo y derecho, seleccionando la mayor de ambas y sumando 1 correspondiente al nodo actual.

Si el árbol está vacío (nullptr), la función retorna 0.

Código:

Hemos implementado esta función que nos permite calcular la altura de un árbol binario. Debemos tomar en cuenta que la altura es el número máximo de niveles desde la raíz hasta la hoja mas profunda. La función recibe un puntero raíz, en el cual pasa por la condición que dice si el nodo este vacío, la altura es 0 y retorna.

Se hace el llamado recursivamente de calcularAltura(), calculamos la altura tanto en derecha como izquierda.

Se toma el máximo de cada altura y se suma mas 1, esto asegura que la altura corresponde al camino mas largo.



```
int calcularAltura(Nodo* raiz) {  
    if (raiz == nullptr) {  
        return 0;  
    }  
  
    int alturaIzquierda = calcularAltura(raiz->izquierdo);  
    int alturaDerecha = calcularAltura(raiz->derecho);  
  
    return 1 + max(alturaIzquierda, alturaDerecha);  
}
```

Ilustración 7. Función calcular altura

Resultado:

El resultado confirma que la función calcularAltura está implementada correctamente, ya que la altura calculada coincide con la esperada en el árbol de prueba. Además, el caso del árbol nulo devuelve 0 como corresponde, validando el manejo del caso base.

```
--- Prueba Ejercicio 3 ---  
Altura esperada: 3  
Altura calculada: 3  
Altura de arbol nulo (esperado 0): 0
```

Ilustración 8.Resultados

Se implementó correctamente el cálculo de la profundidad máxima de un árbol binario utilizando recursividad. El programa logró determinar la altura correcta tanto para un árbol con nodos como para un árbol vacío.

Código JAVA

De la misma forma hemos implementado la misma función de calcular la altura de un árbol en java.

```
public static int calcularAltura(Nodo raiz) {  
  
    if (raiz == null) {  
        return 0;  
    }  
  
    int alturaIzquierda = calcularAltura(raiz.izquierdo);  
  
    int alturaDerecha = calcularAltura(raiz.derecho);  
  
    return 1 + Math.max(alturaIzquierda, alturaDerecha);  
}
```

Ilustración 9.Código JAVA



Usamos la palabra reservada `public`, en el cual es un método estático que calcula la altura del árbol, no usamos los punteros, en cambio usamos `(.)`.

4. Ejercicio 4: Recorridos (Implementación de In-Order).

En este ejercicio se implementó el recorrido In-Order en un árbol binario utilizando recursividad.

El recorrido In-Order visita los nodos en el siguiente orden:

- Subárbol izquierdo.
- Nodo actual.
- Subárbol derecho.

Este tipo de recorrido permite obtener los valores ordenados de menor a mayor cuando se trabaja con un Árbol Binario de Búsqueda (BST).

Código:

Hemos implementado esta función que recibe un puntero raíz y tendrá un vector el cual entra en la condición donde si el nodo es NULL, no tenemos que recorrer y retornamos inmediatamente, este es el caso base de la recursividad.

Llamamos recursivamente a la función `inOrderAux()` que recorre primero el subárbol izquierdo. Se guarda el valor del nodo actual en el vector resultado.

Finalmente recorrer el subárbol derecho.

```
void inOrderAux(Nodo* nodo, vector<int>& resultado) {  
    if (nodo == nullptr) {  
        return;  
    }  
  
    // Recorrer subárbol izquierdo  
    inOrderAux(nodo->izquierdo, resultado);  
  
    // Guardar nodo actual  
    resultado.push_back(nodo->valor);  
  
    // Recorrer subárbol derecho  
    inOrderAux(nodo->derecho, resultado);  
}
```

Ilustración 10. Función recorrido

Resultado:

El recorrido in-order se ejecutó correctamente, ya que la secuencia obtenida coincide exactamente con la esperada. Esto confirma que la función recursiva visitó primero los nodos izquierdos, luego la raíz y finalmente los derechos.



```
--- Prueba Ejercicio 4 ---  
Resultado esperado: 1 2 3 4 5 6 7  
Tu resultado:      1 2 3 4 5 6 7
```

Ilustración 11.Resultados

Se implementó correctamente el recorrido In-Order utilizando recursividad, logrando recorrer todos los nodos del árbol binario en el orden esperado y mostrando los valores de manera ascendente.

Código Java

Se implemento el recorrido inorden en la sintaxis de JAVA.

```
public static void inOrderAux(Nodo nodo, List<Integer> resultado) {  
    if (nodo == null) {  
        return;  
    }  
  
    // Recorrer subárbol izquierdo  
    inOrderAux(nodo.izquierdo, resultado);  
  
    // Guardar nodo actual  
    resultado.add(nodo.valor);  
  
    // Recorrer subárbol derecho  
    inOrderAux(nodo.derecho, resultado);  
}
```

Ilustración 12.Función inorden

Se implemento una función estática llamada inOrderAux, que recibe un nodo y una lista y recorrerá el árbol de la siguiente manera, primero a la izquierda, luego la raíz y por último el subárbol derecho.

5. Ejercicio 5: Transformación (Inversión o árbol espejo).

En este ejercicio se implementó la inversión de un árbol binario, también conocida como árbol espejo.

El algoritmo intercambia los hijos izquierdo y derecho de cada nodo utilizando recursividad.

De esta manera, la estructura original del árbol queda reflejada como un espejo.

Código:

Implementamos la siguiente función, donde recibe un puntero raíz, la condicional dice si la raíz es NULL, no hay nada que invertir y retorna null.

Usamos una variable temporal temp, donde se guarda el hijo izquierdo, luego asignamos el hijo derecho al izquierdo y finalmente asignamos temp al derecho.



Finalmente llamamos recursivamente a la función para invertir izquierdo y derecho y retorna en la raíz invertida.

```
Nodo* invertir(Nodo* raiz) {  
    if (raiz == nullptr) {  
        return nullptr;  
    }  
  
    // Intercambiar hijos  
    Nodo* temp = raiz->izquierdo;  
    raiz->izquierdo = raiz->derecho;  
    raiz->derecho = temp;  
  
    // Invertir subárboles  
    invertir(raiz->izquierdo);  
    invertir(raiz->derecho);  
  
    return raiz;  
}
```

Ilustración 13.Funcion invertir

Resultado:

Se confirma que la función de inversión del árbol binario funciona correctamente, ya que los hijos izquierdo y derecho se intercambiaron como se esperaba.

```
--- Prueba Ejercicio 5 ---  
Antes de invertir:  
Hijo Izq: 2 | Hijo Der: 3  
  
Despues de invertir (Esperado: Izq 3 | Der 2):  
Hijo Izq: 3 | Hijo Der: 2
```

Ilustración 14.Resultado

Se implementó correctamente la inversión de un árbol binario utilizando recursividad, logrando intercambiar los nodos hijos izquierdo y derecho de cada nivel del árbol y obteniendo el comportamiento esperado.

Código JAVA

Implementamos la función invertir em código java.



```
public static Nodo invertir(Nodo raiz) {  
    if (raiz == null) {  
        return null;  
    }  
  
    // Intercambiar hijos  
    Nodo temp = raiz.izquierdo;  
  
    raiz.izquierdo = raiz.derecho;  
  
    raiz.derecho = temp;  
  
    // Invertir subárbol izquierdo  
    invertir(raiz.izquierdo);  
  
    // Invertir subárbol derecho  
    invertir(raiz.derecho);  
  
    return raiz;  
}
```

Ilustración 15. Función invertir

Tenemos la función `invertir`, que recibe el nodo, si el árbol este vacío no recorrerá y terminará. La función cumple en `temp` se guarda el nodo izquierdo. Luego se guarda el derecho y en `temp` recibe el nodo derecho, se llama a la función de `invertir` y se devuelve ya realizado la acción.

Pruebas

Para ver los resultados tras la ejecución de los códigos aplicamos el siguiente comando para c++:

```
PS C:\Users\LENOVO\APEN3-Arboles\cpp> g++ Ejercicio4_Recorridos.cpp -o Cuatro  
PS C:\Users\LENOVO\APEN3-Arboles\cpp> .\Cuatro
```

De la misma manera lo hacemos para java con los siguientes comandos:

```
PS C:\Users\LENOVO\APEN3-Arboles\java> javac Ejercicio5_Transformacion.java Ejercicio2_Binario.java  
PS C:\Users\LENOVO\APEN3-Arboles\java> java Ejercicio5_Transformacion
```

Hacemos uso de la herramienta de Git Bash para clonar el repositorio y hacer commits, podemos ver a continuación el historial de commits, en total hicimos 11 commits.



```
LENOVO@DESKTOP-C5HVSNL MINGW64 ~/APEN3-Arboles (main)
commit b9e3b3c0d97424e58f97ecfb5c70b36b3b1f8774 (HEAD -> main, origin/main, origin/HEAD)
Author: Alexis Nata <alexis2000nata@gmail.com>
Date: Sat May 9 16:34:31 2026 -0500

    Resolucion del ejercicio 5 java

commit a2415c9ff45f7b2a56c97beaed396efbc1e4b91d2
Author: Alexis Nata <alexis2000nata@gmail.com>
Date: Sat May 9 16:29:21 2026 -0500

    Resolucion del ejercicio 4 java

commit 5e61758cfad3e6119664d8225adc989e9bbcaebf
Author: Alexis Nata <alexis2000nata@gmail.com>
Date: Sat May 9 16:21:59 2026 -0500

    Resolucion del ejercicio 3 java

commit 6c9ef846801f6da4e6cf1015518ea0663b6cdaa6
Author: Alexis Nata <alexis2000nata@gmail.com>
Date: Sat May 9 16:10:21 2026 -0500
```

Ilustración 16. Commits

2.8 Habilidades blandas empleadas en la práctica

- ☒ Trabajo en equipo
- ☐ Responsabilidad

2.9 Conclusiones

- Se logró implementar correctamente estructuras de árboles N-arios y árboles binarios en los lenguajes C++ y Java, fortaleciendo la comprensión de las estructuras jerárquicas de datos.
- La utilización de recursividad permitió resolver de manera eficiente problemas relacionados con conteo de nodos, recorridos y cálculo de profundidad en árboles binarios.
- Se comprendió el funcionamiento de los Árboles Binarios de Búsqueda (BST), aplicando correctamente la lógica de inserción y organización de nodos.
- Los recorridos de árboles, especialmente el recorrido In-Order, permitieron visualizar los datos almacenados de manera ordenada y comprender mejor la navegación dentro de la estructura.
- La transformación de árboles mediante la inversión o árbol espejo ayudó a comprender la manipulación de punteros y referencias entre nodos dentro de una estructura dinámica.

2.10 Recomendaciones

- Practicar constantemente la implementación de árboles y estructuras dinámicas para mejorar la lógica de programación y el manejo de recursividad.
- Realizar pruebas con diferentes estructuras y cantidades de nodos para verificar el correcto funcionamiento de los algoritmos implementados.
- Mantener una correcta organización y documentación del código para facilitar su comprensión y mantenimiento.
- Analizar visualmente la estructura de los árboles antes de programar, ya que esto facilita la comprensión de los recorridos y transformaciones.



- Continuar profundizando en temas relacionados con estructuras de datos avanzadas, como árboles AVL, árboles balanceados y grafos, para ampliar los conocimientos en programación y algoritmos.

2.11 Referencias bibliográficas

- [1] “Estructuras de datos II: ARBOLES N-ARIOS.” Accessed: May 08, 2026. [Online]. Available: <https://miblogestructurasdedatos.blogspot.com/p/arboles-n-arios-un-arbol-n-ario-es-una.html>
- [2] “DSA Binary Trees.” Accessed: May 08, 2026. [Online]. Available: https://www.w3schools.com/dsa/dsa_data_binarytrees.php
- [3] “DSA Binary Search Trees.” Accessed: May 08, 2026. [Online]. Available: https://www.w3schools.com/dsa/dsa_data_binarysearchtrees.php

2.12 Anexos

A continuación, colocamos el link del repositorio de la APE, donde hemos resultado cada caso aplicando nuestra lógica en las funciones que requería, el cual se divide en carpetas en donde la resolución de los ejercicios está en carpetas diferentes ya sea para c++ y java.

Link:

<https://github.com/Alexis112008/APEN3-Arboles>